

ECS 408/608

Assignment 1

Introduction to Linux Tools

Solutions

Kunwar Arpit Singh, 22185

For code repository of this assignment: [Operating Systems Assignments](#)

1. Problem 1

Problem statement

With this problem, we will understand the status of the running process using the top command. To do this, compile the program cpu.c and run it on bash or any other terminal of your choice as follows:

```
gcc cpu.c -o cpu
./cpu
```

This program executes an infinite loop. Now open another window of the terminal screen and, run the top command, and answer the following question about the cpu process.

- Determine the PID of the process executing the CPU program.
- How much memory and CPU does this process consume?
- Determine the current status of the process. For instance, is it executing in a blocked, active, or zombie state?

Answers

- (a) PID of the process: 105690
- (b) Memory and CPU consumption: 100.0% and 0.0%
- (c) Current Status: Running as denoted by R in the terminal => Active

Output Screenshots

Terminal output

```
top - 16:05:52 up 19:08, 1 user, load average: 1.01, 1.01, 1.00
Tasks: 57 total, 2 running, 55 sleeping, 0 stopped, 0 zombie
Mem: 317.8 total, 8.3 free, 8.4 used, 79.4 free, 0.1 used, 0.0 free, 0.0 st
Mem Mem: 7765.2 total, 4869.3 free, 2312.8 used, 752.4 buff/cache
Mem Swap: 2848.0 total, 2848.0 free, 0.0 used, 5453.0 avail mem

PID USER      PP  NI  VIRT  RES  SHR  S  %CPU  %MEM    time+  COMMAND
105690 kas453  20   0 2532  1824 1824  R 100.0  0.0 190.09u 0.00  cpu
96  root      20   0 24872  4896 4902  S  0.3  0.1  0:04.34  systemd-udevd
3682 kas453  20   0 1875824 89440 48128  S  0.3  1.0  2:03.35  node
1700 kas453  20   0 1354880 81560 45680  S  0.3  1.1  0:25.95  node
187438 kas453  20   0 1811504 52480 44288  S  0.3  0.7  0:03.28  node
2  root      20   0 21760  1552  924  S  0.2  0.0  0:09.75  systemd
2  root      20   0 3128  1520  1520  S  0.0  0.0  0:00.56  init-systemd(ub
2  root      20   0 3128  1792  1792  S  0.0  0.0  0:00.00  init
43  root     19   1 58424  1792 16896  S  0.0  0.2  0:08.57  systemd-journal
200 systemd+  20   0 21504  1072 18096  S  0.0  0.2  0:01.02  systemd-resolve
117 systemd+  20   0 61824  7680 6784  S  0.0  0.1  0:03.53  systemd-timesyn
150 root     20   0 4236  3080  2432  S  0.0  0.0  0:00.44  cron
157 message+  20   0 5636  4864 4480  S  0.0  0.1  0:01.92  dbus-daemon
164 root     20   0 1768  8640  7532  S  0.0  0.1  0:04.43  systemd-logind
170 root     20   0 1904528 14768 11264  S  0.0  0.2  0:06.45  selinux.service
170 syslog   20   0 22256  5632 4080  S  0.0  0.1  0:02.07  syslogd
181 root     20   0 3168  3080  1920  S  0.0  0.0  0:00.03  sgetty
184 root     20   0 3116  1920  1792  S  0.0  0.0  0:00.00  sgetty
190 root     20   0 167032 20520 13112  S  0.0  0.3  0:00.16  unattended-upgr
282 root     20   0 6696  4224  3712  S  0.0  0.1  0:00.01  login
372 kas453  20   0 70312  1136 968  S  0.0  0.1  0:00.01  systemd
330 kas453  20   0 21152  3520  1792  S  0.0  0.0  0:00.00  (sd-pam)
353 kas453  20   0 4892  5276  3584  S  0.0  0.1  0:00.01  bash
3459 polkitd  20   0 388164 7680 6592  S  0.0  0.1  0:02.94  polkitd
3649 root     20   0 3176  96 768  S  0.0  0.0  0:00.00  systemd-udevd
3650 root     20   0 3164  1168 1824  S  0.0  0.0  0:00.02  relay(3653)
3653 kas453  20   0 2880  1664  1664  S  0.0  0.0  0:00.01  sh
3653 kas453  20   0 2880  1792  1792  S  0.0  0.0  0:00.00  sh
3650 kas453  20   0 2880  1792  1792  S  0.0  0.0  0:00.00  sh
3662 kas453  20   0 11.3g 13264 53760  S  0.0  1.5  1:10.93  node
3734 kas453  20   0 73.6g 67620 65236  S  0.0  8.5  2:07.41  node
3765 kas453  20   0 6280  5240  3584  S  0.0  0.1  0:00.06  bash
3800 root     20   0 3128  96 160  S  0.0  0.0  0:00.00  systemd-udevd
3807 root     20   0 3164  1168 1824  S  0.0  0.0  0:00.01  relay(3808)
3800 kas453  20   0 3880  1664  1664  S  0.0  0.0  0:00.00  sh
3809 kas453  20   0 2880  1792  1792  S  0.0  0.0  0:00.00  sh
3809 kas453  20   0 2880  1792  1792  S  0.0  0.0  0:00.00  sh
3846 kas453  20   0 2880  1792  1792  S  0.0  0.0  0:00.01  sh
3896 kas453  20   0 6156  5016  3520  S  0.0  0.1  0:14.53  perl
3911 kas453  20   0 6280  4992  3456  S  0.0  0.1  0:00.03  bash
4143 kas453  20   0 185608 48964 42780  S  0.0  2.1  23:08.42  python
182743 kas453  20   0 756876 66340 16640  S  0.0  0.1  0:03.35  python
```

2. Problem 2

Problem statement

In this problem, you are going to understand the hardware configuration of your Linux machine using the `/proc` filesystem.

Execute command `more /proc/cpuinfo` and explain the following terms: cores and processors. Furthermore, use the command `lscpu` to verify your definitions. Before attempting this question, you need to understand the concept of CPU hyperthreading at a high level.

- How many cores does your machine have?
- How many processors does your machine have?
- Determine the frequency of each processor.
- Determine how much physical memory your system has.
- Determine the architecture of CPU?
- How much of the memory is occupied?
- Determine the total number of forks and context switches since your system booted up.

Answers

- (a) Number of Cores: 2
- (b) Number of Processors: 1
- (c) Frequency of each processor: 2995.20
- (d) Total Physical Memory: 7951600 kB
- (e) CPU Architecture: x86_64
- (f) Occupied Memory: $\text{MemTotal} - \text{MemAvailable} = 7951600 \text{ kB} - 4973764 \text{ kB} = 2977836 \text{ kB}$
- (g) Forks: 108857
- Context Switches: 38243277

Output Screenshots

Terminal output

```
hash453@LAPTOP-BUHEH377:~/ent/d/Code&Study/Program Files/IISER Books/Sem8 Books/Operating Systems/Assignments/Assignment 1$ more /proc/cpuinfo
processor       : 0
vendor_id     : GenuineIntel
cpu family    : 6
model         : 140
model name    : 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
stepping      : 1
microcode    : 0xffffffff
cpu MHz       : 2995.200
cache size    : 6144 KB
physical id   : 0
siblings      : 4
core id       : 0
cpu cores     : 2
apicid        : 0
initial apicid : 0
fpu           : yes
fpu_exception : yes
cpuid level   : 27
wp            : yes
flags         : fpu vme de pse msr pae mce cx8 apic sep mtrr pge nca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp
lm constant_tsc arch_perfmon rep_good nopl stoplead tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pdcm pci
d sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs
enhanced tpr_shadow ept_vpid ept_ad fsgsbase tsc_adjust bml avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma clflushopt
clwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves vnni avx512vbmi umip avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512
bitnl avx512_vpoperdtd rdpid movdiri movdir64b fsrm avx512_vdintersect md_clear flush_lid arch_capabilities
vmx flags     : vnni invvpid ept_x_only ept_ad ept_lgb tsc_offset vtptr ept vpid unrestricted_guest ept_mode_based_exec tsc_scaling
bugs          : spectre_v1 spectre_v2 spec_store_bypass swapgs retbleed e1bs_pbrsb bhi
bogomips      : 5999.48
clflush size  : 64
cache_alignm  : 64
address sizes : 39 bits physical, 48 bits virtual
power management:

processor      : 1
vendor_id     : GenuineIntel
cpu family    : 6
model         : 140
model name    : 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
```

Terminal output

```

ksw453@APT0P:~$ cat /proc/meminfo
MemTotal:       7951680 kB
MemFree:        4973764 kB
MemAvailable:   5578436 kB
Buffers:        55560 kB
Cached:         653876 kB
SwapCached:      0 kB
Active:         278384 kB
Inactive:       2333652 kB
Active(anon):    2784 kB
Inactive(anon): 1896232 kB
Active(file):    267520 kB
Inactive(file): 437420 kB
Unevictable:     0 kB
Mlocked:        0 kB
SwapTotal:      2697152 kB
SwapFree:       2697152 kB
Dirty:          24 kB
Writeback:       0 kB
AnonPages:      1895348 kB
Mapped:         245432 kB
Shmem:          3716 kB
KReclaimable:   62332 kB
Slab:           115464 kB
SReclaimable:   62332 kB
SUnreclaim:     53132 kB
KernelStack:    5856 kB
PageTables:     32928 kB
SecPageTables:   0 kB
NFs_Unstable:    0 kB
Bounce:         0 kB
WritebackTmp:    0 kB
CommitLimit:    6072952 kB
Committed_AS:   2529680 kB
VmallocTotal:   34359738367 kB
VmallocUsed:     38832 kB
VmallocChunk:    0 kB
Percpu:         1696 kB
HardwareCorrupted: 0 kB
AnonHugePages:   0 kB
ShmemHugePages:  0 kB
ShmemPmdMapped:  0 kB
FileHugePages:   0 kB
FileMapped:      0 kB
Unaccepted:      0 kB
HugePages_Total: 0
HugePages_Free:  0
HugePages_Rsvd:  0
HugePages_Surp:  0
Hugepagesize:    2048 kB
Hugetlb:         0 kB
DirectMap4k:    74752 kB
DirectMap2M:    4982784 kB
DirectMap1G:    11534336 kB

```

Terminal output

[illegible]

Terminal output

```

root@kali:~# apt-get install -y socat; socat file:`hostname`:/dev/null tcp:localhost:8080 && sleep 10m; $ lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 39 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 4
On-line CPU(s) llist: 0-3
Vendor ID: GenuineIntel
Model name: 11th Gen Intel(R) Core(TM) i3-1115G6 @ 3.80GHz
CPU family: 6
Model: 140
Thread(s) per core: 2
Core(s) per socket: 2
Socket(s): 2
Stepping: 1
BogoMIPS: 5990.40
Flags: fpu_vme pse tsc msr pae mce cx8 apic sep mtrr pge nca cmov pat pse36 clflush rmx fxsr sse sse2 ss ht syscall nx pdpeibg rdtsge
in_constant_tsc arch_perfmon rep_good noopl xtology_tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx smx sse4f frma ext3
6 pdcn pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch stio
bld ibrs tpbp sttbp lbrs enhanced_tpr_shadow ept vpid ept-ad fsgsbase tsc_adjust bti avx2 smep bmi2 erms invpcid avxsiz2f avxsiz2dq
rdseed adx snap avxsiz2fno_clflushopt clwb avxsiz2cd sha_ni avxsiz2bw avxsiz2v1_xsaveopt xsavec xgetbv1 xsavevm avxsiz2vml unip_a
vsiz2_vbnrt_gpi waitpkg vpcmqlqdq avxsiz2_vml avxsiz2_bitql avxsiz2_vpcompdq rapid_model1 model-qrb fsm avxsiz2_vp2intersect no_cl
flush_l3arch_capabilities

Virtualization features:
VT-x: On
Hypervisor vendor: Microsoft
Virtualization type: full
Caches (sum of all):
L1d: 96 KiB (2 instances)
L1i: 64 KiB (2 instances)
L2: 2.5 MiB (2 instances)
L3: 6 MiB (1 instance)
NUMA:
NUMA node(s): 1
Local NUMA node CPU(s): 0-3
Vulnerabilities:
gather data sampling: Not affected
itlb multihit: Not affected
l1tf: Not affected
mds: Not affected
 meltdown: Not affected
 mitty state data: Not affected
rog file data sampling: Not affected
retbleed: Mitigation; Enhanced IBRS
spec rstack overflow: Not affected
spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
spectre v1: Mitigation; usercopy/swaps barriers and user pointer sanitization
spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; RSB filling; PBRSSB-eIBRS SW sequence; BHI SW loop, KVM SW loop
sbds: Not affected
tsx async abort: Not affected

```

3. Problem 3

Problem statement

Compile and execute the two programs `memory_1.c` and `memory_2.c` given to you. These programs allocate a large array in memory. The only difference is one program accesses the array, and the other doesn't. Note that both programs pause before exiting to let you inspect their memory usage. With the `ps` command, you can inspect the memory used by these programs. For instance, the output will tell you the total size of the “virtual” memory of the process is, and how much of this is actually physically resident in memory. With that, you will learn the virtual memory allocated to the process that the process thinks it has. Whereas the OS only allocates a subset of this memory physically in RAM. Compare both programs' virtual and physical memory usage, and explain your observations.

Answers

Comparison Table:

Program	Virtual Memory (VSZ)	Resident Set Size (RSS)
<code>memory_1.c</code> (Accessed)	6468	5376
<code>memory_2.c</code> (Not Accessed)	6472	5376

Explanation: The metrics obtained via the `ps` utility illustrate the fundamental distinction between the logical address space perceived by a process and the physical memory resources managed by the Operating System kernel.

- **Virtual Memory Size (VSZ):** Both programs exhibit a similar Virtual Memory footprint, approximately 6.4 MB. This value represents the total logical address space mapped for the process. It accounts for the large array requested via allocation, regardless of whether that memory is actively utilized.
- **Resident Set Size (RSS):** The RSS identifies the portion of the process's memory that is currently held in physical RAM. In this experimental run under the WSL environment, both programs registered an identical RSS of 5376 KB.
- **Analysis of Results:** Modern operating systems typically implement *demand paging*, a strategy where the kernel defers the mapping of physical frames until a page fault is triggered by a memory access. Under this theoretical framework, `memory_2.c` (which allocates but does not access the array) should exhibit a significantly lower RSS than `memory_1.c`.

However, the observed parity in RSS suggests environmental factors at play within the WSL/Linux subsystem. This behavior may be attributed to a “greedy” allocation policy where the system pre-commits physical pages upon request, or it may indicate that the total memory footprint of the test programs falls within the minimum allocation granularity of this specific kernel configuration. Consequently, the OS has committed the physical pages for the array even in the absence of explicit read/write operations by the user program.

Output Screenshots

Terminal output

```
kas4453@LAPTOP-8U4EH377: /mnt/d/Code&Study/Program Files/IISER Books/Sem8 Books/Operating Systems/Assignments/Assignment : $ gcc memory_1.c -o memory_1
kas4453@LAPTOP-8U4EH377: /mnt/d/Code&Study/Program Files/IISER Books/Sem8 Books/Operating Systems/Assignments/Assignment : $ ./memory_1

Program : 'memory_1'
-----

PID : 108587
Size of int : 4

Press Enter Key to exit.
█
```

Terminal output

```
kas4453@LAPTOP-8U4EH377: /mnt/d/Code&Study/Program Files/IISER Books/Sem8 Books/Operating Systems/Assignments/Assignment : $ ps -o pid,vsz,rss,comm -p $(pgrep memory_1)
  PID  VSZ  RSS COMMAND
108564 6468 5376 memory_1
█
```

Terminal output

```
kas4453@LAPTOP-8U4EH377: /mnt/d/Code&Study/Program Files/IISER Books/Sem8 Books/Operating Systems/Assignments/Assignment : $ gcc memory_2.c -o memory_2
kas4453@LAPTOP-8U4EH377: /mnt/d/Code&Study/Program Files/IISER Books/Sem8 Books/Operating Systems/Assignments/Assignment : $ ./memory_2

Program : 'memory_2'
-----

PID : 108610
Size of int : 4

Press Enter Key to exit.
█
```

Terminal output

```
kas4453@LAPTOP-8U4EH377: /mnt/d/Code&Study/Program Files/IISER Books/Sem8 Books/Operating Systems/Assignments/Assignment : $ ps -o pid,vsz,rss,comm -p $(pgrep memory_2)
  PID  VSZ  RSS COMMAND
108610 6472 5376 memory_2
█
```

4. Problem 4

Problem statement

In this problem, you will understand how the Linux shell (e.g., bash shell) executes user commands by spawning new child processes to execute the various commands.

(a) Compile the program `cpu-print.c` given to you and execute it in the bash or any other shell of your choice as follows.

```
gcc cpu-print.c -o cpu-print
./cpu-print
```

This program executes in an infinite loop, printing output to the screen. Now, open another terminal and use the `ps` command with suitable options to find out the `pid` of the process spawned by the shell to run the `cpu-print` executable. You may want to explore the `ps` command thoroughly to understand its various output fields.

(b) Find the PID of the parent of the `cpu-print` process, i.e., the shell process. Next, find the PIDs of all the ancestors, going back at least five generations (or until you reach the limit process).

(c) You will now understand how the shell performs output redirection. Execute the following command.

```
./cpu-print > /tmp/tmp.txt &
```

Look at the `proc` file system information of the newly spawned process. Please pay particular attention to where its file descriptors 0, 1, and 2 (standard input, output, and error) are pointing to. Using this information, can you describe how Shell is implementing I/O redirection?

(d) With this subproblem, you will learn how to implement pipes using the shell. For this, execute the following command:

```
./cpu-print | grep hello &
```

One more time, find the newly spawned processes and look out for the standard input/output. Error file descriptors are pointing to; with this, explain how pipes are implemented in the shell.

(e) When you type a command in the shell, the shell performs two things. For some commands, binary files that perform that functionality already come with your Linux kernel installation. For such an operation, the shell invokes the binary like it executes the binaries of your own programs. The shell implements the command within its code for other commands where the executable or binaries do not exist.

Consider the following command that you can type in bash shell: `cd`, `ls`, `history`, `ps`. Determine which of these commands already exist as executables in the Linux kernel directory tree, which are then simply executed by the bash shell and implemented by the bash code itself.

Answer

(a) PID of the `cpu-print` process

Using the command:

```
ps -ef | grep cpu-print
```

the PID of the running `cpu-print` process was identified as **5575**.

(b) Parent and ancestor processes

The parent PID of the `cpu-print` process was determined using:

```
ps -fp 5575
```

The process hierarchy is as follows:

- **5575**: `cpu-print`
- **3393**: `bash (shell)`
- **3281**: `gnome-terminal-server`
- **1503**: `systemd -user`
- **1**: `init (root process)`

This confirms that the shell spawns the `cpu-print` process and that all user processes ultimately trace back to the `init` process.

(c) Output redirection

The command executed was:

```
./cpu-print > /tmp/tmp.txt &
```

Inspecting the file descriptors using:

```
ls -l /proc/5666/fd
```

revealed:

- File descriptor 0 (`stdin`) points to `/dev/pts/0`
- File descriptor 1 (`stdout`) points to `/tmp/tmp.txt`
- File descriptor 2 (`stderr`) points to `/dev/pts/0`

This shows that the shell performs output redirection by modifying the `stdout` file descriptor of the child process before execution, redirecting it from the terminal to the specified file.

(d) Pipes

The following command was executed:

```
./cpu-print | grep hello &
```

The shell creates a pipe and connects:

- `stdout` of `cpu-print` to `stdin` of `grep`
- `stdin` and `stderr` remain connected to the terminal

This is implemented by the shell using the `pipe()`, `fork()`, and `dup2()` system calls to connect the output of one process to the input of another.

(e) Built-in vs external commands

Command	Implementation
cd	Shell built-in
ls	External executable (/bin/ls)
history	Shell built-in
ps	External executable (/bin/ps)

Commands like `cd` must be implemented within the shell since they modify the shell's internal state, whereas others exist as standalone binaries.

Output Screenshots

Terminal output

```
iiserb@ccclab4:~/os$ ps -ef | grep cpu-print
iiserb      5575    3393  61 12:30 pts/0    00:00:10 ./cpu-print
iiserb      5611    5577   0 12:30 pts/1    00:00:00 grep --color=auto cpu-print
```

Terminal output

```
iiserb@ccclab4:~/os$ ps -ef | grep cpu-print
iiserb      5575    3393  61 12:30 pts/0    00:00:10 ./cpu-print
iiserb      5611    5577   0 12:30 pts/1    00:00:00 grep --color=auto cpu-print
iiserb@ccclab4:~/os$ ps -fp 3393
UID        PID     PPID  C STIME TTY          TIME CMD
iiserb      3393    3281   0 12:19 pts/0    00:00:00 bash
iiserb@ccclab4:~/os$ ps -fp 3281
UID        PID     PPID  C STIME TTY          TIME CMD
iiserb      3281    1503  14 12:19 ?        00:01:54 /usr/libexec/gnome-terminal-server
iiserb@ccclab4:~/os$ ps -fp 1503
UID        PID     PPID  C STIME TTY          TIME CMD
iiserb      1503       1   0 11:11 ?        00:00:01 /lib/systemd/systemd --user
iiserb@ccclab4:~/os$ ps -fp 1
UID        PID     PPID  C STIME TTY          TIME CMD
root         1         0   0 11:11 ?        00:00:01 /sbin/init splash
iiserb@ccclab4:~/os$ ps -o pid,ppid,cmd -p 3393
PID     PPID  CMD
3393    3281  bash
iiserb@ccclab4:~/os$ ps -o pid,ppid,cmd -p 3281
PID     PPID  CMD
3281    1503  /usr/libexec/gnome-terminal-server
iiserb@ccclab4:~/os$ ps -o pid,ppid,cmd -p 1503
PID     PPID  CMD
1503     1    /lib/systemd/systemd --user
iiserb@ccclab4:~/os$ ps -o pid,ppid,cmd -p 1
PID     PPID  CMD
1        0    /sbin/init splash
```

Terminal output

```
iiserb@ccclab4:~/os$ ./cpu-print > /tmp/tmp.txt &
[1] 5666
```

Terminal output

```
iiserb@ccclab4:~/os$ ./cpu-print > /tmp/tmp.txt &
[1] 5666
iiserb@ccclab4:~/os$ ls -l /proc/5666/fd
total 0
lrwx----- 1 iiserb iiserb 64 Jan 27 12:34 0 -> /dev/pts/0
l-wx----- 1 iiserb iiserb 64 Jan 27 12:34 1 -> /tmp/tmp.txt
lrwx----- 1 iiserb iiserb 64 Jan 27 12:34 2 -> /dev/pts/0
iiserb@ccclab4:~/os$ readlink /proc/5666/fd/1
/tmp/tmp.txt
```

5. Problem 5

Problem statement

In this problem, you will compile and execute the program `disk\_1.c` and `disk\_2.c`. These programs read a large number of files from the disk. To do so, you first create these files as follows: Create a folder `disk-files` and place `foo.pdf` file in it. Consequently, use the script `make-copies.sh` (place outside the `disk-files` folder) to make 5000 copies of the same file in folder `disk-files`. The disk programs will read these files. Now, execute the disk programs one after another. With each program, measure the disk utilization while the program is executing on the system. To measure the disk utilization, use the `iostat` tool (You can find the usage of `iostat` with `man iostat` command). Report and explain all your observations. Along with that, read the code of the program in order to explain your observation.

Hint: Note that for this exercise to work correctly, you must be reading from a directory on the local disk. If your `disk-files` directory is not on a local disk (but, say, mounted via NFS), then you must alter the location of the files in the code provided to you to enable reading from a local disk. Also, modern operating systems store recently read files in a cache in memory (called disk buffer cache) for faster access to the same files in the future. In order to ensure that you are making observations while actually reading from disk, you must clear your disk buffer cache between multiple runs of `disk_1.c`. If you do not clear the disk buffer cache between successive runs of `disk_1.c`, you will be reading the files not from disk but from memory. Look up online for commands on how to clear your disk buffer cache, and note that you will need superuser permissions to execute these commands.

Answers

A directory containing approximately 5000 copies of the same file was created and read by the programs `disk_1.c` and `disk_2.c`. Disk utilization was monitored using `vmstat` during execution.

Observations:

- During the first execution, a noticeable number of block input operations (`bi`) were observed, indicating active disk reads.
- Subsequent executions showed reduced disk activity due to the Linux disk buffer cache.
- CPU wait time (`wa`) remained low, suggesting that the disk subsystem was not a bottleneck.

Explanation: The Linux kernel aggressively caches disk blocks in memory. Once the files are read during the first execution, subsequent reads are served from memory rather than disk, significantly reducing I/O overhead.

The observed behavior aligns with the implementation of `disk_1.c` and `disk_2.c`, which sequentially read multiple files. Without clearing the disk buffer cache, later executions do not accurately represent disk performance.

This experiment highlights the importance of cache effects in disk I/O benchmarking.

Output Screenshots

Terminal output

```
iiserb@cclab4:~/os$ ls disk-files | wc -l
1
iiserb@cclab4:~/os$ ls disk-files | wc -l
4526
iiserb@cclab4:~/os$ ls disk-files | wc -l
5002
iiserb@cclab4:~/os$ gcc disk_1.c -o disk_1
gcc disk_2.c -o disk_2
iiserb@cclab4:~/os$ df -T disk-files
Filesystem      Type 1K-blocks      Used Available Use% Mounted on
/dev/nvme0n1p8 ext4 125634356 79893256  39288100  68% /
```

Terminal output

```
iiserb@cclab4:~/os$ mkdir disk-files
iiserb@cclab4:~/os$ cp foo.pdf disk-files/
iiserb@cclab4:~/os$ chmod +x make-copies.sh
./make-copies.sh
```

Terminal output

```
iiserb@cclab4:~/os$ vmstat 1
procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
r  b   swpd   free   buff   cache   si   so    bi    bo    in   cs  us  sy  id  wa  st
2  0     256 2518464 786692 9242044    0    0    43   994   333  178 15   3  82   0   0
2  0     256 2518212 786692 9242044    0    0    0    0   3101 1881 11 15  74   0   0
2  0     256 2517712 786692 9241980    0    0    0    0   3356 2689 13 14  73   0   0
2  0     256 2516716 786692 9241980    0    0    0   196 3842 3119 14 16  70   0   0
2  0     256 2516716 786692 9241980    0    0    0    4 3178 2178 12 14  74   0   0
2  0     256 2516716 786692 9241980    0    0    0   72 3165 1916 12 14  74   0   0
2  0     256 2525028 786700 9236816    0    0    0   52 3021 1371 11 14  74   0   0
2  0     256 2525028 786700 9236752    0    0    0    0 3190 1860 12 15  74   0   0
2  0     256 2524776 786700 9236752    0    0    0    0 2744 1028 11 14  75   0   0
2  0     256 2524776 786700 9236752    0    0    0    0 2937 1259 11 14  75   0   0
3  0     256 2516724 786700 9244120    0    0    0    0 4475 4495 14 15  70   0   0
3  0     256 2471880 786700 9277572    0    0    0   44 6237 6541 28 15  56   1   0
2  0     256 2481192 786700 9270224    0    0    0    0 4245 4818 14 15  71   0   0
2  0     256 2489652 786708 9265752    0    0    0   180 4077 4704 13 15  72   0   0
2  0     256 2485872 786708 9265560    0    0    0    0 4150 5757 15 15  71   0   0
3  0     256 2485380 786708 9265624    0    0    0    0 4637 5552 19 15  66   0   0
2  0     256 2483124 786708 9267032    0    0    0    0 4794 5654 13 15  72   0   0
2  0     256 2485364 786708 9267164    0    0    0    0 7181 7073 20 15  64   0   0
6  0     256 2485820 786716 9265884    0    0    0   140 4859 4867 14 15  71   0   0
3  0     256 2483388 786716 9266468    0    0    0    4 4743 5630 16 14  69   0   0
3  0     256 2477340 786716 9266352    0    0    0    0 5176 5359 16 16  68   0   0
2  0     256 2469704 786716 9271452    0    0    0    0 3903 3821 16 15  70   0   0
2  0     256 2464916 786716 9268620    0    0    0   184 4376 2966 18 14  67   0   0
2  0     256 2451016 786716 9277532    0    0    0    0 5402 6392 17 15  67   1   0
2  0     256 2447236 786716 9275676    0    0    0   144 3531 2628 12 15  73   0   0
2  0     256 2447984 786724 9272604    0    0    0   20 4976 5753 16 15  69   0   0
2  0     256 2447984 786724 9272592    0    0    0    0 3043 1405 11 14  75   0   0
```

Terminal output

```

iiserb@cclab4:~/os$ vmstat 1
procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs us  sy id wa st
1  0     256 2338132 786820 9257428    0    0    42   982   334  181 15   3  82   0  0
1  0     256 2342712 786820 9257428    0    0    0     0  1901 1667   6   7  87   0  0
1  0     256 2349256 786820 9252308    0    0    0     0  1791 1224   6   7  87   0  0
1  0     256 2353632 786820 9252308    0    0    0     0  1644 1074   6   7  87   0  0
1  0     256 2350056 786820 9257508    0    0    0     0  2103 2299   6   7  87   0  0
1  0     256 2349584 786820 9257852    0    0    0    12 3692 5120   8   9  83   0  0
1  0     256 2337848 786820 9275192    0    0    0     0  4920 6637  16   8  75   1  0
1  0     256 2343332 786820 9273156    0    0    0     0  3758 5396   7   8  84   0  0
2  0     256 2372732 786820 9272048    0    0    0     0  3657 5433   7   9  85   0  0
1  0     256 2359448 786820 9283352    0    0    0     0  3811 5538   8   8  85   0  0
1  0     256 2358780 786820 9278488    0    0    0     0  3525 5017   7   8  85   0  0
1  0     256 2353768 786828 9283236    0    0    0    12 3965 6400   8   8  84   0  0
1  0     256 2353604 786828 9285284    0    0    0     0  3122 4161   8   8  84   0  0
1  0     256 2355500 786828 9279764    0    0    0     0  2069 1697   6   8  87   0  0
1  0     256 2355500 786828 9269524    0    0    0     0  1930 1233   6   7  87   0  0
1  0     256 2355700 786828 9269372    0    0    0     0  2499 1891   6   8  86   0  0
1  0     256 2346124 786836 9281024    0    0    0    36 4333 6105  10   9  81   0  0
1  0     256 2342072 786836 9280756    0    0    0     0  2210 1537   5   8  87   0  0
1  0     256 2345388 786836 9275344    0    0    0     0  2303 1964   6   8  86   0  0
1  0     256 2347392 786836 9266900    0    0    0     0  2075 1433   6   7  87   0  0

```

Terminal output

```

iiserb@cclab4:~/os$ vmstat 1
procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs us  sy id wa st
1  0     256 2348216 786892 9256880    0    0    42   976   334  182 15   3  82   0  0
1  0     256 2347140 786892 9257968    0    0    0     0  1975 1498   6   6  87   0  0
1  0     256 2347124 786892 9258108    0    0    0     8 2552 2828   7   7  85   0  0
1  0     256 2336256 786892 9270984    0    0    0     0  4340 6948  11   7  81   0  0
1  0     256 2336144 786900 9269064    0    0    0    12 3344 4282   9   7  83   1  0
3  0     256 2359212 786908 9263700    0    0    0    28 5539 10087  16   8  76   0  0
3  0     256 2361516 786908 9258180    0    0    0     0  4790 8862  14   8  77   0  0
2  1     256 2361972 786908 9254004    0    0    0     0  3332 5978  10   8  82   0  0
3  0     256 2378620 786916 9254004    0    0    0    24 5622 6649  16   8  75   0  0
1  0     256 2376736 786916 9245812    0    0    0     8 3500 5175   9   7  84   0  0
1  0     256 2376512 786916 9245684    0    0    0   308 2164 3385   7   7  86   0  0
1  0     256 2376704 786916 9245748    0    0    0     0  3542 6083  10   8  82   0  0
1  0     256 2375620 786916 9245748    0    0    0     0  3582 6477  10   8  82   0  0
1  0     256 2370328 786916 9245748    0    0    0     0  2428 3728   8   7  85   0  0
1  0     256 2342332 786924 9245748    0    0    0    32 3438 5584  12   7  82   0  0
1  0     256 2333660 786924 9240688    0    0    0     0  3296 5185  10   7  83   0  0
2  0     256 2339112 786924 9241200    0    0    0     0  3195 5421  10   7  83   0  0
2  0     256 2338516 786924 9241200    0    0    0     0  4012 6555  12   7  81   0  0
1  0     256 2324276 786924 9240688    0    0    0   120 3671 6074  11   7  82   0  0
1  0     256 2327984 786924 9240624    0    0    0     8 3948 6765  11   8  81   0  0
2  0     256 2350372 786932 9240628    0    0    0   360 5729 6929  17   8  75   0  0
1  0     256 2327580 786932 9241716    0    0    0     0  4391 6993  11   8  81   0  0
1  0     256 2321200 786932 9241780    0    0    0     0  3704 6469  11   8  82   0  0
1  0     256 2317824 786932 9240756    0    0    0     0  2757 4467   8   7  85   0  0
1  0     256 2323720 786932 9240692    0    0    0     0  3577 6136  10   8  82   0  0
1  0     256 2323388 786940 9240696    0    0    0    60 3084 5241   9   7  84   0  0
3  0     256 2323392 786940 9240760    0    0    0   292 2574 4079   8   7  85   0  0
1  0     256 2321688 786940 9240760    0    0    0     0  2225 2939   7   7  86   0  0
2  0     256 2323852 786940 9240824    0    0    0     0  3879 6161  11   7  82   0  0
1  0     256 2332720 786940 9240824    0    0    0     0  3560 5883  10   7  82   0  0
2  0     256 2314028 786948 9240696    0    0    0    28 2839 4766   9   7  84   0  0
1  0     256 2313508 786948 9256156    0    0    0     4 5602 10097  15   8  77   0  0
3  0     256 2315468 786948 9253580    0    0    0     0  3495 5996  10   7  83   0  0
1  0     256 2310352 786948 9256308    0    0    0     0  4190 6591  13   8  80   0  0

```

6. Problem 6

Problem statement

Different user programs make several system calls (which are most likely the function call to OS, but more complicated) to invoke OS functionality during their execution. The `strace` tool let you trace the system calls invoked by a running executable. Consider any executable in the example above or any Linux command like `ls` and execute with the `strace` command as follows:

```
strace <executable>
```

This command shows the list of all system calls made by the executable during its run. How many system calls does the program make in total? Look up online to find the list of system calls supported by your operating system, and see if you can locate any of them in the output of `strace` command.

Answers

The `strace` utility was used to trace system calls made by the `ls` command:

```
strace ls
```

Observations:

- The command invoked approximately **120 system calls** during execution.
- Common system calls observed include:
 - `execve` – to execute the binary
 - `openat` – to open directories and files
 - `read` – to read directory contents
 - `fstat` – to retrieve file metadata
 - `mmap` and `brk` – for memory management
 - `close` – to release file descriptors

Conclusion: Even a simple command like `ls` relies on numerous system calls to interact with the kernel. This demonstrates how user-space programs depend heavily on kernel services for file I/O, memory management, and process control.

Output Screenshots

Terminal output

```
lsuser@cclab:~/os$ strace l
execve("/usr/bin/lis", ["lis"], 0x7fffff41902c0 /* 49 vars */) = 0
brk(NULL)                                = 0x645d0a3c1000
arch_prctl(0x3001 /* ARCH_???, */ 0x7ffe1bb07740) = -1 EINVAL (Invalid argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x71303ead5000
access("/etc/ld.so.preload", R_OK)       = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfststat(3, "", {st_mode=S_IFREG|0644, st_size=90536, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 90536, PROT_READ, MAP_PRIVATE, 3, 0) = 0x71303eabe000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0-\0\1\0\0\0\0\0\0\0\0\0\0....", 832) = 832
newfststat(3, "", {st_mode=S_IFREG|0644, st_size=166280, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 177672, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x71303ea92000
mprotect(0x71303ea98000, 139264, PROT_NONE) = 0
mmap(0x71303ea98000, 106496, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x6000) = 0x71303ea98000
mmap(0x71303eab2000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x20000) = 0x71303eab2000
mmap(0x71303eaba000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x27000) = 0x71303eaba000
mmap(0x71303eabc000, 5640, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x71303eabc000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0-\0\1\0\0\0\0\0\0\0\0\0\0....", 832) = 832
pread64(3, "{\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0....", 784, 64) = 784
pread64(3, "{\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0....", 84, 848) = 48
pread64(3, "{\4\0\0\0\0\24\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0....", 68, 896) = 68
newfststat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0
pread64(3, "{\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0....", 784, 64) = 784
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x71303e800000
mprotect(0x71303e828000, 2023424, PROT_NONE) = 0
mmap(0x71303e828000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x71303e828000
mmap(0x71303e9bd000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b000) = 0x71303e9bd000
mmap(0x71303ea16000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x215000) = 0x71303ea16000
mmap(0x71303ea1c000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x71303ea1c000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcr2-2.8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0-\0\1\0\0\0\0\0\0\0\0\0\0....", 832) = 832
newfststat(3, "", {st_mode=S_IFREG|0644, st_size=613064, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 615184, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x71303e769000
mmap(0x71303e76b000, 438272, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) = 0x71303e76b000
mmap(0x71303e7d6000, 163840, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x6d000) = 0x71303e7d6000
mmap(0x71303e7fe000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x94000) = 0x71303e7fe000
close(3)                                 = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x71303ea8f000
arch_prctl(ARCH_SET_FS, 0x71303ea8f800) = 0
set_tid_address(0x71303ea8fad0)          = 19022
set_robust_list(0x71303ea8fae0, 24)      = 0
rseq(0x71303ea901a0, 0x20, 0, 0x53053053) = 0
mprotect(0x71303ea16000, 16384, PROT_READ) = 0
mprotect(0x71303e7fe000, 4096, PROT_READ) = 0
mprotect(0x71303eaba000, 4096, PROT_READ) = 0
```

Terminal output

```

protect(0x645d08aa5000, 8192, PROT_READ) = 0
mprotect(0x7c5d3eb0f000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
mmap(0x71303eabe000, 90536) = 0
stats("/sys/fs/selinux", 0x7ffe1bb07780) = -1 ENOENT (No such file or directory)
stats("/selinux", 0x7ffe1bb07780) = -1 ENOENT (No such file or directory)
getrandom("\xf1\xf6\x5a\xd2\x37\x35\x9d\x73", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x645d0a3c1000
brk(0x645d0a3e2000) = 0x645d0a3e2000
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0444, st_size=0, ...}, AT_EMPTY_PATH) = 0
read(3, "\0dev\tsysfs\n\0dev\ttmpfs\n\0dev\tbd...", 1024) = 393
close(3) = 0
close(3) = 0
access("/etc/selinux/config", F_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=5712208, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 5712208, PROT_READ, MAP_PRIVATE, 3, 0) = 0x71303e000000
close(3) = 0
ioctl(1, TCGETS, {B38400 opost isig icanon echo ...}) = 0
ioctl(1, TIOCGWINSZ, {ws_row=50, ws_col=132, ws_xpixel=0, ws_ypixel=0}) = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
newfstatat(3, "", {st_mode=S_IFDIR|0775, st_size=4096, ...}, AT_EMPTY_PATH) = 0
getdents64(3, 0x645d0a3c79f0 /* 14 entries */, 32768) = 448
getdents64(3, 0x645d0a3c79f0 /* 0 entries */, 32768) = 0
close(3) = 0
newfstatat(1, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x1), ...}, AT_EMPTY_PATH) = 0
write(1, "cpu-print disk_1 disk_2\t d...", 70cpu-print disk_1 disk_2 disk-files make-copies.sh memory_2.c) = 70
write(1, "cpu-print.c disk_1.c disk_2.c ...", 75cpu-print.c disk_1.c disk_2.c foo.pdf memory_1.c strace_ls.txt) = 75
close(1) = 0
close(2) = 0
exit_group(0) = ?
+++ exited with 0 +++

```